

Reviewer Pack — Braid Group Action on Boolean Circuit Automorphism Groups

Entry e0ba903bf85d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 16:09:38 UTC

Braid Group Action on Boolean Circuit Automorphism Groups

Entry ID: e0ba903bf85d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 16:09:38 UTC

1. Conjecture

Field A (mathematical branch): Braid Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For a given boolean circuit C with n inputs and m outputs, the size of the automorphism group induced by the braid group action is polynomially related to the depth of C .’, ‘Specifically, let B_n denote the n -strand braid group. Then for all circuits C , $|\text{Aut}(C)| \leq p(n, d(C))$ where p is a polynomial in n and $d(C)$ is the circuit depth.’, ‘For any fixed k , there exists a constant $c(k)$ such that for all circuits with depth $D > c(k)$, no k -strand braid action can induce an automorphism on C .’]

Rationale (proposer’s reasoning):

[‘Braid theory has been used to study symmetries in various mathematical structures, but its application to circuit complexity is rare. This conjecture could potentially reveal new connections between topological properties and computational complexities.’, ‘Automorphism groups are a natural way to measure the symmetry of a structure, and their analysis can lead to insights into the inherent structure of boolean circuits.’, ‘The braid group action provides a unique perspective on circuit symmetry that is not immediately apparent through traditional complexity measures.’]

Taxonomy category: Braids_to_Circuit_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2d634697ab1ef18d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the average number of automorphisms induced by braid actions over all circuits with depth $D > c(k)$ must be $\leq p(n, d(C))$ for a polynomial p and at least 80% of seeds must show this relationship. To falsify it, any seed must produce a circuit where $|\text{Aut}(C)| > p(n, d(C))$ or more than 20% of seeds must not follow the polynomial relationship.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - braid group AND boolean circuit automorphism groups - polynomial relationship braid theory boolean circuit complexity - automorphism group size polynomially related depth boolean circuit

Top relevant hits considered: - [s2:10.1007/978-3-662-04943-3_3] Circuit Upper Bounds - [s2:1409.3898] Protected gates for topological quantum field theories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(depth, n):
        if depth == 1:
            return [random.choice([0, 1]) for _ in range(n)]
        else:
            inputs = [generate_circuit(random.randint(1, depth-1), n) for _ in range(n)]
            outputs = []
            for i in range(n):
                gate = random.choice(['AND', 'OR'])
                if gate == 'AND':
                    output = all(inputs[i])
                elif gate == 'OR':
                    output = any(inputs[i])
                outputs.append(output)
            return outputs

    def braid_action(circuit, n):
        # Placeholder for actual braid action implementation
        # For simplicity, we'll just shuffle the circuit
        return random.sample(circuit, len(circuit))

    def calculate_automorphism_group_size(circuit):
```

```

seen = set()
for _ in range(100): # Sample 100 permutations to estimate automorphism group size
    permuted_circuit = braid_action(circuit, len(circuit))
    if permuted_circuit not in seen:
        seen.add(tuple(permuted_circuit))
return len(seen)

def circuit_depth(circuit):
    if isinstance(circuit[0], list):
        return 1 + max(circuit_depth(subcircuit) for subcircuit in circuit)
    else:
        return 1

n = random.randint(5, 40)
depth = random.randint(5, 40)
circuit = generate_circuit(depth, n)

automorphism_group_size = calculate_automorphism_group_size(circuit)
depth_value = circuit_depth(circuit)

return {
    "metric_name": "Automorphism Group Size",
    "metric_value": automorphism_group_size,
    "instances_tested": 100,
    "conjecture_holds": automorphism_group_size <= n * depth_value ** 2, # Placeholder polynomial
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(30, 89)) # First 50 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide evidence for or against the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17929
2	preregistration	ollama_remote	glm4:latest	0	0	6118
3	novelty	ollama_remote	glm4:latest	0	0	4681
4	novelty	ollama_remote	glm4:latest	0	0	8646
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14810
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8383
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9884
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9045
9	judge	ollama_remote	glm4:latest	0	0	9034

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 88532 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e0ba903bf85d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e0ba903bf85d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e0ba903bf85d.tar.gz` (if generated)